

Gradient Descent Variants: Batch vs Stochastic vs Mini-Batch

Comparison Table

Feature	Batch GD	Stochastic GD	Mini-Batch GD
Updates per epoch	1	N times	$\frac{N}{B}$ times
Memory requirement	High	Low	Moderate
Convergence speed	Slow	Fast	Moderate
Stability	High (smooth)	Low (noisy)	Moderate
Parallel Processing	Poor	Good	Best

1. Batch Gradient Descent (BGD)

- Updates weights after computing the gradient for the entire dataset.

python

Copy Edit

```
for epoch in range(num_epochs): # Loop over multiple passes through the dataset
    total_gradient = 0 # Accumulate gradients
    for i in range(N): # Loop over all training samples
        gradient = compute_gradient(X[i], y[i], weights) # Compute gradient for sample i
        total_gradient += gradient # Accumulate gradient

    total_gradient /= N # Average over all samples
    weights -= learning_rate * total_gradient # Update weights

    loss = compute_loss(X, y, weights) # Compute loss for monitoring
    print(f"Epoch {epoch+1}, Loss: {loss}")
```

- Update happens after the full dataset is processed.
- Slower but stable convergence ↓

2. Stochastic Gradient Descent (SGD)

- Updates weights after each training sample, leading to faster updates but more noise.

python

Copy Edit

```
for epoch in range(num_epochs): # Loop over multiple passes through the data
    for i in range(N): # Loop over all training samples
        gradient = compute_gradient(X[i], y[i], weights) # Compute gradient for sample i
        weights -= learning_rate * gradient # Update weights

    loss = compute_loss(X, y, weights) # Compute loss after epoch
    print(f"Epoch {epoch+1}, Sample {i+1}, Loss: {loss}")
```

- Frequent updates lead to high variance in weights.
- May require learning rate decay to stabilize convergence.

3. Mini-Batch Gradient Descent (MBGD)

- Uses a subset of data (batch) before updating weights.

```
python Copy Edit

batch_size = B # Set the mini-batch size
for epoch in range(num_epochs): # Loop over multiple passes t
    for j in range(0, N, batch_size): # Loop over dataset in
        batch_X = X[j:j+batch_size] # Select batch of inputs
        batch_y = y[j:j+batch_size] # Select batch of outputs

        gradient = compute_gradient(batch_X, batch_y, weights)
        weights -= learning_rate * gradient # Update weights

    loss = compute_loss(X, y, weights) # Compute loss for mon
    print(f"Epoch {epoch+1}, Loss: {loss}")
```



Key Differences in Update Frequency

Method	Weight Update
Batch GD	Once per epoch
SGD	After each sample
Mini-Batch GD	After every batch